

École Polytechnique de Ouagadougou
Filière Génie informatique

Rapport de travaux pratiques

Ingénierie de la cryptographie
Module EC1 : TP 1, TP 2 et TP 3

Étudiant : SAWADOGO Abdoul Mohaimine Wendoldé

Enseignant : Dr Zakaria Sawadogo

Année : 2026–2027

Date : 31 octobre 2026

1 TP 1 : chiffrement authentifié et dérivation de clés

1.1 Dérivation des clés et format

Le module `coffre.py` ne réimplémente aucune primitive. `cle_maitresse` dérive une clé de 32 octets du mot de passe avec Argon2id et un sel de 16 octets propre à chaque fichier. `sous_cle` en tire ensuite une clé par usage avec HKDF-SHA256 et `info = b"coffre-v1|" + usage`.

Mesuré sur ma machine (moyenne de trois essais), $(t, m, p) = (3, 64Mio, 4)$ prend 0,10 s, $(3, 128Mio, 4)$ 0,19 s et $(4, 256Mio, 4)$ 0,47 s. Ce dernier jeu se rapproche de la cible de 0,5 s. Le code garde le premier par défaut, les paramètres restant réglables et inscrits dans chaque fichier.

Le fichier `CFR1` commence par un en-tête de 44 octets : l'identifiant `CFR1` (4 octets, format et version), les paramètres (t, m, p) en trois entiers de 32 bits (12 octets, pour redériver la clé), le sel (16 octets), le nonce (12 octets). Viennent ensuite le chiffré et l'étiquette GCM de 16 octets.

Les données associées sont l'en-tête complet suivi du nom du fichier : authentifiés sans être chiffrés. Au déchiffrement, le module refuse un blob trop court ou un identifiant inconnu (`ValueError`), relit (t, m, p) dans l'en-tête, vérifie qu'ils restent dans des bornes raisonnables ($t \leq 10$, $m \leq 1$ Gio, $p \leq 16$) et recalcule la clé. Une altération ou un mauvais mot de passe lève `InvalidTag`.

1.2 Tests

Quatorze tests `pytest` passent (figure 1). Les cinq propriétés demandées y sont : aller-retour, chiffrement probabiliste (sels et nonces différents), mauvais mot de passe, octet modifié, nom de fichier échangé. S'y ajoutent les données vides et les paramètres par défaut, la modification d'un octet de l'en-tête (paramètre, sel, nonce), le rejet de paramètres démesurés, le rejet d'un format inconnu, le rechiffrement et la séparation des sous-clés.

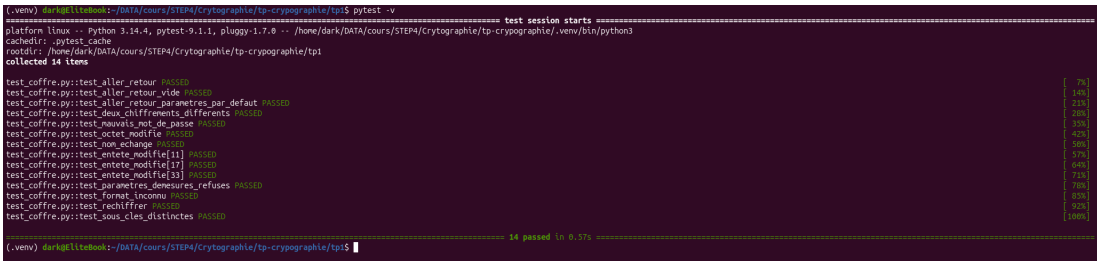


FIGURE 1. `pytest -v` sur `test_coffre.py`.

1.3 Évolution

Le format permet de durcir Argon2id sans rendre les anciens fichiers illisibles, car (t, m, p) est écrit dans l'en-tête de chacun : un ancien fichier se déchiffre avec ses paramètres d'origine, un nouveau utilise les paramètres renforcés, et le code de lecture ne change pas. `rechiffrer(ancien_mdp, nouveau_mdp, nom, blob)` déchiffre avec l'ancien mot de passe puis rechiffre avec le nouveau, un sel et un nonce neufs, et les paramètres demandés.

1.4 Questions de réflexion

Question 1. *Pourquoi le sel est-il stocké en clair ?*

Il doit être unique, pas secret. Il empêche le précalcul de dictionnaires valables pour tous les utilisateurs et donne deux clés différentes à deux fichiers de même mot de passe. La sécurité repose sur le coût mémoire et temps d'Argon2id.

Question 2. *Pourquoi aucun risque de répétition du nonce ?*

AES-GCM interdit de répéter un couple (clé, nonce). Le sel étant neuf à chaque fichier, la clé l'est aussi : deux

nonces identiques ne se rencontreraient jamais sous la même clé. Chaque clé ne chiffre qu'un message, donc la limite d'anniversaire des nonces de 96 bits ne joue pas.

Question 3. *Et si le nom n'était pas dans les données associées ?*

Le chiffré ne serait plus lié à son identité. Si `salaires.xlsx` et `menu.txt` sont chiffrés avec le même mot de passe, un employé malveillant échange les noms : le déchiffrement réussit et il lit un contenu qui ne lui était pas destiné. Avec le nom authentifié, `InvalidTag` bloque l'attaque.

Question 4. *Pourquoi authentifier l'en-tête ?*

Sans cela, un attaquant pourrait modifier les paramètres Argon2id pour les abaisser et affaiblir la dérivation, ou changer le sel et le nonce pour rendre le fichier illisible sans que personne sache pourquoi. Avec l'en-tête dans les données associées, l'étiquette échoue dès qu'un bit change. Un point demande pourtant de la prudence : les paramètres sont lus avant que l'étiquette puisse être vérifiée, puisqu'il faut la clé pour la vérifier. Un attaquant qui écrit une mémoire ou un nombre d'itérations démesurés ferait donc tourner Argon2id avant d'être détecté (déni de service). C'est pourquoi `dechiffrer` refuse d'abord les paramètres hors bornes.

Question 5. *Pourquoi ne pas détailler la raison d'un échec ?*

Des messages distincts (mot de passe faux, nom différent, étiquette corrompue) font de l'application un oracle qui renseigne l'attaquant. On affiche un seul message, sans donnée partielle.

2 TP 2 : infrastructure de gestion de clés

2.1 Architecture et configuration

L'architecture suit l'énoncé : une AC racine (P-384, 10 ans, `pathlen:1`, hors ligne), une AC intermédiaire (P-256, 5 ans, `pathlen:0`, contraintes de nom limitées à `tp.local`), un certificat de serveur pour `www.tp.local` et deux certificats de client, alice et bob (P-256, 90 jours, `CA:FALSE`).

Dans chaque fichier de configuration, la section de l'autorité indique où `openssl ca` trouve ses éléments : `database` (registre des certificats émis ou révoqués), `serial` et `crlnumber` (prochains numéros), `default_md` (hachage), `default_crl_days` (durée de validité d'une CRL), `policy` (champs du nom exigés) et `copy_extensions` : `none` pour la racine, qui ignore les extensions demandées dans une CSR, `copy` pour l'intermédiaire, qui reprend le SAN de la demande. Les extensions se lisent ainsi.

— `ext_inter` (racine). `basicConstraints = critical, CA:TRUE, pathlen:0` : une AC qui ne signe que des feuilles ; `critical` oblige le vérificateur à comprendre l'extension, sinon il rejette. `keyUsage = critical, keyCertSign, cRLSign` : la clé ne signe que des certificats et des CRL. `subjectKeyIdentifier` et `authorityKeyIdentifier` permettent de reconstruire la chaîne. `nameConstraints = critical, permitted;DNS:.tp.local, permitted;DNS:tp.local, permitted;email:tp.local` : l'intermédiaire ne peut certifier que ce domaine.
— `ext_serveur` : `CA:FALSE` (jamais une AC), `keyUsage = digitalSignature` (suffisant pour une clé EC, pas de chiffrement de clé), `extendedKeyUsage = serverAuth` (usage serveur TLS), et les deux identifiants de clé comme plus haut.
— `ext_client` : les mêmes lignes, avec `extendedKeyUsage = clientAuth` (usage client TLS).

`unique_subject = no` permet de réémettre un certificat pour un même sujet, ce qui sert au renouvellement du TP 3.

2.2 Autorités et certificats

La racine est un certificat auto-signé de 3650 jours sur une clé EC P-384, avec les extensions critiques `CA:TRUE`, `pathlen:1` et `Certificate Sign, CRL Sign`. L'émetteur et le sujet sont identiques (`CN=Racine TP`) : personne au-dessus ne peut la signer, sa confiance vient de son installation dans le magasin du client. L'intermédiaire est signé par la racine pour 1825 jours ; on y retrouve `pathlen:0` et les contraintes de nom (figure 2). Le serveur (SAN `www.tp.local`, `tp.local`) et les clients alice et bob génèrent chacun leur clé et sont signés 90 jours par l'intermédiaire. `pki.sh` regroupe le tout et commence par `set -euo pipefail`, donc s'arrête à la première erreur. Le certificat pour `www.example.com` et la révocation d'alice (étapes 5 et 6) sont regroupés dans `revocation.sh`.



FIGURE 2. Certificat de l'intermédiaire : `pathlen:0` et contraintes de nom.

2.3 Validations

```
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt -verify_ho
stname www.tp.local serveur.crt
serveur.crt: OK
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt -verify_ho
stname autre.example serveur.crt
CN=www.tp.local
error 62 at 0 depth lookup: hostname mismatch
error serveur.crt: verification failed
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt -purpose s
slclient alice.crt
alice.crt: OK
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt -verify_ho
stname www.example.com example.crt
CN=www.example.com
error 47 at 0 depth lookup: permitted subtree violation
error example.crt: verification failed
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$
```

FIGURE 3. Résultats de `openssl verify`.

La figure 3 se lit ainsi. Le serveur est accepté pour `www.tp.local`. Pour `autre.example`, la chaîne est valide mais le nom est absent du SAN (`hostname mismatch`). Alice est acceptée avec l'usage `sslclient`. Le certificat demandé pour `www.example.com` échoue avec `permitted subtree violation`.

L'intermédiaire a pu signer ce dernier certificat parce que `openssl ca` applique sa configuration locale sans contrôler ses propres contraintes. C'est le client qui, en construisant la chaîne (RFC 5280), applique les contraintes de nom à tout ce que l'intermédiaire a signé. Une AC mal configurée ou compromise reste ainsi confinée à son domaine.

2.4 Révocation

La clé d'alice est supposée compromise : le certificat est révoqué avec le motif `keyCompromise`, puis les CRL de l'intermédiaire et de la racine sont générées et regroupées dans `crls.pem`. Avec `-crl_check_all`, la vérification d'alice échoue avec `certificate revoked`, alors que bob reste accepté (figure 4).

```
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl verify -crl_check_all -CRLfile crls.pem -CAfile racine/racine.crt -unt
rusted inter/inter.crt alice.crt
CN=alice@tp.local
error 23 at 0 depth lookup: certificate revoked
error alice.crt: verification failed
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl crl -in inter/crl/inter.crl -noout -text | head -25
Certificate Revocation List (CRL):
        Version 2 (0x1)
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: C=BF, O=Universite TP, CN=Inter TP
        Last Update: Oct 31 20:14:37 2026 GMT
        Next Update: Nov  7 20:14:37 2026 GMT
        CRL extensions:
            X509v3 CRL Number:
                4096
Revoked Certificates:
    Serial Number: 1001
        Revocation Date: Oct 31 20:14:37 2026 GMT
        CRL entry extensions:
            X509v3 CRL Reason Code:
                Key Compromise
    Signature Algorithm: ecdsa-with-SHA256
    Signature Value:
        30:45:02:21:00:c2:27:f0:64:5d:b6:ec:30:30:4f:9f:e0:73:
        00:46:8a:6d:a9:67:8b:24:9e:dd:1c:4c:2a:84:cc:45:31:48:
        b4:02:20:5c:38:1a:9f:03:ab:b8:aa:77:f7:5c:06:ce:4e:1c:
        87:92:4b:22:25:1c:de:34:10:17:2d:db:f0:90:12:ed:23
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$
```

FIGURE 4. Alice avec les CRL, et contenu de la CRL de l'intermédiaire.

2.5 Questions de réflexion

Question 1. *Pourquoi la racine reste-t-elle hors ligne ? Intermédiaire ou racine compromise ?*

La racine est l'ancre de toute la chaîne : si sa clé fuit, tout ce qu'elle a certifié devient suspect. Comme elle intervient rarement, on la garde déconnectée, sur support chiffré ou HSM, en coffre. Si l'intermédiaire est compromis, on ressort la racine pour le révoquer sur sa CRL, on crée une nouvelle intermédiaire et on réémet les feuilles ; les clients gardent la même racine. Si la racine est compromise, elle ne peut pas se révoquer elle-même : il faut bâtir une nouvelle PKI et remplacer l'ancre de confiance partout.

Question 2. *À quoi sert `pathlen:0` ? Quel risque si on l'oublie ?*

Il interdit à l'intermédiaire de signer d'autres autorités. Sans cette limite, quiconque détient sa clé pourrait créer des sous-AC, qui en créeraient d'autres, et la délégation échapperait à tout contrôle.

Question 3. *Pourquoi le serveur génère-t-il sa propre clé ?*

Une clé privée ne doit ni circuler ni être connue d'un tiers. Si l'AC la générait, ses opérateurs pourraient déchiffrer le trafic ou se faire passer pour le serveur. Le demandeur n'envoie que la clé publique, dans la CSR.

Question 4. *CN ou SAN ?*

Les clients modernes vérifient le SAN. Le CN n'est plus utilisé pour la vérification de nom depuis la RFC 6125 (qui reprend la RFC 2818), et plusieurs navigateurs rejettent un certificat dont le nom n'est pas dans une entrée `DNS:`.

Question 5. *Limites des CRL, comparaison avec OCSP et les certificats courts.*

Mécanisme	**Avantage**	**Limites**
CRL	simple, utilisable hors ligne	liste qui grossit, à jour seulement à la publication suivante
OCSP	statut d'un seul certificat, quasi temps réel	latence, panne du répondeur, il apprend les sites visités (l'agrafage limite ce point)
Courte durée	pas de révocation à gérer, exposition bornée	exige un renouvellement automatique fiable

TABLE 1. Mécanismes de révocation.

Question 6. *Politique de certification minimale.*

1. Elle ne couvre que les serveurs et personnes internes du domaine `tp.local`.
2. Le nom d'un serveur est vérifié dans la zone DNS interne avant signature.
3. L'identité d'un utilisateur est contrôlée par l'annuaire de l'organisation.
4. Les clés sont générées par les demandeurs, en EC P-256 au minimum, et jamais transmises.
5. Durées : 10 ans (racine), 5 ans (intermédiaire), 90 jours (feuilles).
6. Le SAN est obligatoire et l'intermédiaire porte `pathlen:0`.
7. Une compromission se déclare sous 4 heures, la CRL est republiée sous 1 heure.
8. La clé racine reste hors ligne, en coffre, et sa réactivation exige deux personnes habilitées.
9. Les expirations sont surveillées chaque jour, toute demande hors profil est rejetée.
10. Les supports de clés sont détruits de façon certifiée en fin de vie.

3 TP 3 : canal sécurisé et gestion des certificats

3.1 Configuration nginx

Le serveur présente sa chaîne (son certificat suivi de l'intermédiaire, jamais la racine) : `cat serveur.crt inter/inter.crt > serveur_chaine.crt`. La clé privée est en mode 600, les certificats et la CRL en 644, et `www.tp.local` est ajouté à `/etc/hosts` (le tout est fait par `deployer.sh`).

```
server {
    listen 443 ssl;
    server_name www.tp.local;
    ssl_certificate     /etc/nginx/tls/serveur_chaine.crt;
    ssl_certificate_key /etc/nginx/tls/serveur.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_prefer_server_ciphers off;
    ssl_ciphers ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-ECDSA-CHACHA20-POLY1305;
    ssl_session_tickets off;
    ssl_client_certificate /etc/nginx/tls/racine.crt;
    ssl_verify_client on;
    ssl_verify_depth 2;
    ssl_crl /etc/nginx/tls/crls.pem;
    add_header Strict-Transport-Security "max-age=31536000" always;
    location / {
        return 200 "Bonjour $ssl_client_s_dn\n";
    }
}
```

Justification des directives :

- `listen 443 ssl` et `server_name` : HTTPS sur le port standard, pour le nom porté par le certificat.
- `ssl_certificate` et `ssl_certificate_key` : la chaîne à présenter (serveur puis intermédiaire) et la clé privée correspondante.
- `ssl_protocols TLSv1.2 TLSv1.3` : on écarte SSL, TLS 1.0 et 1.1, dépréciés.
- `ssl_prefer_server_ciphers off` : avec des suites modernes, le client choisit celle qui convient le mieux à son matériel (AES accéléré ou ChaCha20).
- `ssl_ciphers` : limite TLS 1.2 aux suites à chiffrement authentifié et confidentialité persistante (voir la vérification plus bas). Le certificat étant ECDSA, seules les suites ECDSA sont utiles.
- `ssl_session_tickets off` : évite qu'une clé de ticket jamais renouvelée ruine la confidentialité persistante.
- `ssl_client_certificate` : la racine qui doit avoir signé les certificats clients.
- `ssl_verify_client on` : le client doit présenter un certificat, c'est l'authentification mutuelle.
- `ssl_verify_depth 2` : au plus deux niveaux de chaîne (intermédiaire puis racine).
- `ssl_crl` : le serveur consulte la liste de révocation à chaque connexion.
- `Strict-Transport-Security` : le navigateur n'utilisera que HTTPS pendant un an.
- `return 200 ...` : renvoie le sujet du certificat client authentifié.

3.2 Poignée de main

Avec le certificat de bob, la connexion s'établit en TLS 1.3, suite `TLS_AES_256_GCM_SHA384`, groupe `X25519MLKEM768` (hybride post-quantique), vérification du serveur `OK` (figure 5). La commande `curl` de l'énoncé, avec `bob.crt` seul, est refusée (400) : nginx ne connaît que la racine et ne peut pas rattacher bob à elle sans l'intermédiaire. Il faut présenter la chaîne du client (bob puis l'intermédiaire), ce que `s_client` fait avec `-cert_chain`.

```
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ curl --cacert racine/racine.crt --cert bob_chaine.crt --key bob.key https://ww
w.tp.local/
openssl s_client -connect www.tp.local:443 -servername www.tp.local \
  -verify_hostname www.tp.local -CAfile racine/racine.crt \
  -cert bob.crt -key bob.key -cert_chain inter/inter.crt \
  -brief </dev/null 2>&1 | grep -v Requested
Bonjour CN=bob@tp.local
Connecting to 127.0.0.1
CONNECTION ESTABLISHED
Protocol version: TLSv1.3
Ciphersuite: TLS_AES_256_GCM_SHA384
Peer certificate: CN=www.tp.local
Hash used: SHA256
Signature type: ecdsa_secp256r1_sha256
Verification: OK
Verified peername: www.tp.local
Negotiated TLS1.3 group: X25519MLKEM768
DONE
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$
```

FIGURE 5. Connexion de bob avec `curl` et `openssl s_client`.

La figure 6 montre trois refus : sans certificat (`No required SSL certificate was sent`), avec le certificat révoqué d'alice (numéro de série trouvé dans la CRL), et en TLS 1.1 (`no protocols available`).

```
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ curl -s --cacert racine/racine.crt https://www.tp.local/
<html>
<head><title>400 No required SSL certificate was sent</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>No required SSL certificate was sent</center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ curl -s --cacert racine/racine.crt --cert alice_chaine.crt --key alice.key htt
ps://www.tp.local/
<html>
<head><title>400 The SSL certificate error</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>The SSL certificate error</center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ openssl s_client -connect www.tp.local:443 -tls1_1 -brief </dev/null
Connecting to 127.0.0.1
4077CB5874730000:error:0A0000BF:SSL routines:tls_setup_handshake:no protocols available:../ssl/statem/statem_lib.c:155:
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$
```

FIGURE 6. Refus sans certificat, avec un certificat révoqué, et en TLS 1.1.

Dans Wireshark (figures 7 et 8), le ClientHello (SNI `www.tp.local`, paquet 4) et le ServerHello (paquet 6) sont lisibles, mais aucun message `Certificate` n'apparaît. En TLS 1.2 il passait en clair juste après le ServerHello. En TLS 1.3, dès le ServerHello, client et serveur ont calculé le secret partagé grâce aux parts d'échange de clés éphémères et dérivé des clés de poignée de main avec HKDF. Tout ce qui suit (`EncryptedExtensions`, `Certificate`, `CertificateVerify`, `Finished`) est chiffré et s'affiche en `Application Data`.

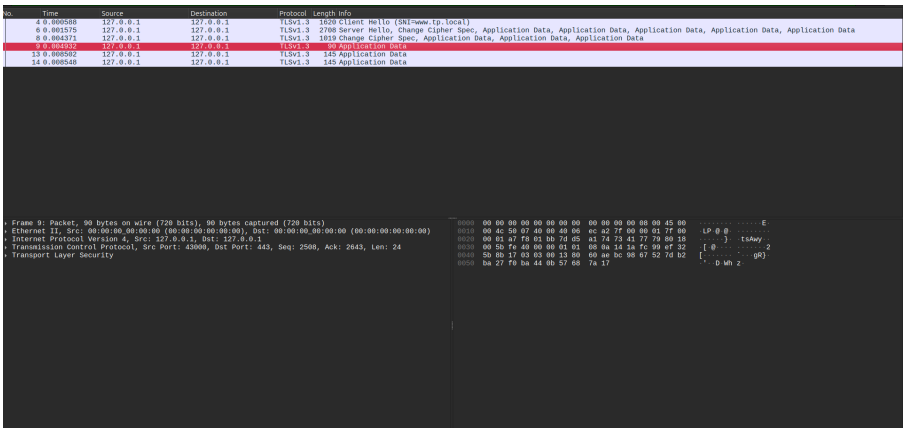


FIGURE 7. ClientHello, ServerHello, puis uniquement des `Application Data`.

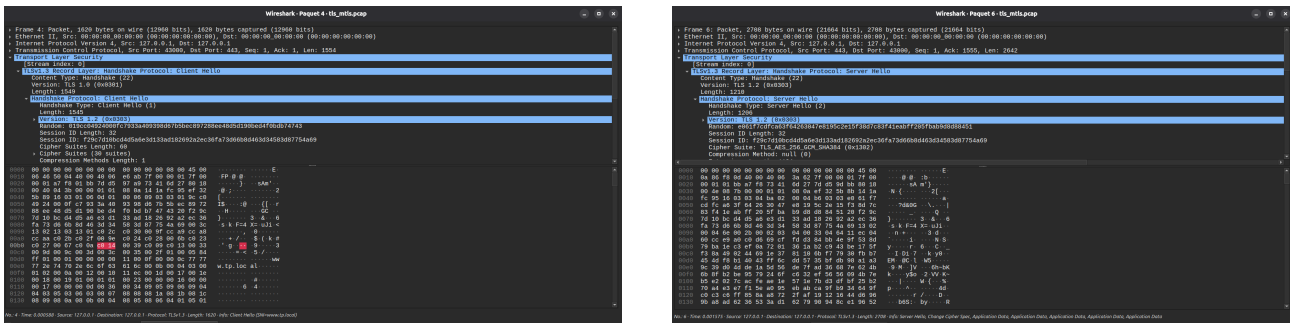


FIGURE 8. ClientHello (paquet 4, 30 suites proposées) et ServerHello (paquet 6, suite retenue).

3.3 Vérification, supervision et renouvellement

Un premier balayage `nmap` a montré en TLS 1.2 des suites en mode CBC, sans chiffrement authentifié. J'ai corrigé avec `ssl_ciphers` : il ne reste que trois suites TLS 1.2, toutes AEAD avec ECDHE, plus les trois suites TLS 1.3 (figure 9).

```
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$ nmap --script ssl-enum-ciphers -p 443 www.tp.local
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-31 20:54 +0000
Nmap scan report for www.tp.local (127.0.0.1)
Host is up (0.00016s latency).
rDNS record for 127.0.0.1: localhost

PORT    STATE SERVICE
443/tcp open  https
| ssl-enum-ciphers:
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (secp256r1) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (secp256r1) - A
|       TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (secp256r1) - A
|     compressors:
|       NULL
|     cipher preference: client
|   TLSv1.3:
|     ciphers:
|       TLS_AKE_WITH_AES_128_GCM_SHA256 (X25519MLKEM768) - A
|       TLS_AKE_WITH_AES_256_GCM_SHA384 (X25519MLKEM768) - A
|       TLS_AKE_WITH_CHACHA20_POLY1305_SHA256 (X25519MLKEM768) - A
|     cipher preference: client
|_  least strength: A

Nmap done: 1 IP address (1 host up) scanned in 0.19 seconds
dark@EliteBook:~/DATA/cours/STEP4/Crytographie/tp-crypographie/tp2$
```

FIGURE 9. `nmap -script ssl-enum-ciphers` après correction.

`surveiller.py` lit tous les certificats d'un dossier et affiche pour chacun les jours restants et un niveau : OK, ALERTE (moins de 30 jours), CRITIQUE (moins de 7 jours ou expiré). Le code de sortie vaut 0, 1 ou 2 selon le pire cas. Testé sur trois dossiers : certificats de 90 jours, code 0 ; un certificat de 20 jours, `ALERTE`, code 1 ; un certificat de 5 jours, `CRITIQUE`, code 2.

`renouveler.sh` lit la date d'expiration du certificat installé dans `/etc/nginx/tls` (`openssl x509 -checkend`) et s'arrête s'il vaut encore plus de 30 jours ; l'option `-force` passe outre. Sinon il génère une nouvelle clé et une demande aux mêmes SAN, la fait signer par l'intermédiaire, puis vérifie la chaîne avec `openssl verify` : en cas d'échec le script s'arrête avant de toucher au serveur. Il copie ensuite l'ancienne clé et l'ancienne chaîne dans `ancien/`, installe les nouveaux fichiers, teste avec `nginx -t` et recharge nginx sans interruption. Si le test échoue, les anciens fichiers sont remis en place. Un service et un minuteur systemd le lancent chaque jour à 3 h (équivalent cron : `0 3 * * *`). Le test a été fait avec `-force` : nginx sert ensuite le nouveau certificat et l'ancien reste disponible.

3.4 Questions de réflexion

Question 1. *Pourquoi ne pas envoyer la racine ?*

Le client doit déjà l'avoir dans son magasin de confiance : il ne ferait pas confiance à une racine reçue à la volée, donc l'envoyer ne sert à rien et alourdit la poignée de main.

Question 2. *Pourquoi désactiver les tickets de session ?*

Un ticket contient les secrets de la session, chiffrés par une clé du serveur. Si cette clé ne change pas, un attaquant qui a enregistré le trafic et finit par la voler déchiffre toutes les sessions passées : la confidentialité

persistante est perdue. Sans tickets, chaque connexion refait un échange ECDHE éphémère.

Question 3. *Que garantit HSTS ?*

Le navigateur transforme lui-même toute adresse `http://` en `https://` avant d'envoyer quoi que ce soit, et ne laisse plus passer outre une erreur de certificat. Cela protège contre le SSL stripping : sur un Wi-Fi public, un attaquant intercepte la première requête en clair et garde la victime en HTTP.

Question 4. *Certificats de 100 jours (2027) puis 47 jours (2029).*

Renouveler à la main devient ingérable : générer, valider et déployer toutes les trois ou quatre semaines sur tout un parc. Un oubli provoque une panne par certificat expiré. L'automatisation (ACME ou scripts planifiés comme ici) devient obligatoire, avec test et supervision.

Question 5. *Pourquoi une chaîne incomplète passe chez certains clients seulement ?*

Les navigateurs de bureau complètent la chaîne : ils gardent en cache les intermédiaires déjà vus ou téléchargent celui qui manque grâce à l'extension AIA. Les clients stricts (cURL, OpenSSL, bibliothèques Python ou Java) ne le font pas et échouent avec `unable to get local issuer certificate`. Le serveur doit donc toujours envoyer sa chaîne complète, sans la racine.

Question 6. *Procédure si la clé privée du serveur fuit.*

1. Contenir : isoler la source de la fuite, prévenir le responsable sécurité, conserver les preuves.
2. Révoquer le certificat avec le motif `keyCompromise`.
3. Publier la CRL mise à jour (et rafraîchir les statuts OCSP s'il y en a).
4. Générer une nouvelle paire de clés sur une machine saine, sans réutiliser l'ancienne, et obtenir un nouveau certificat.
5. Déployer la nouvelle chaîne, tester avec `nginx -t`, recharger.
6. Analyser les journaux pour repérer une usurpation, corriger la cause, rédiger un retour d'expérience.